

TUGAS PRAKTIKUM 4

Nama : Muhammad 'Azmi Salam

NIM : 2406010

Kelas : C2

Tugas 1

Contoh Kasus: Sistem Verifikasi Integritas Dokumen Akademik

Sebuah universitas perlu memastikan bahwa dokumen-dokumen akademik yang diterbitkan (transkrip nilai, ijazah, surat keterangan) tidak dapat dipalsukan atau dimodifikasi oleh pihak yang tidak berwenang. Sistem ini menggunakan fungsi hashing untuk mengamankan dokumen-dokumen tersebut.

a. Data apa yang akan di-hash?

Data yang akan di-hash adalah konten dokumen akademik berupa string teks yang merepresentasikan isi dokumen, seperti:

- Transkrip Nilai: berisi identitas mahasiswa (nama, NIM), IPK, dan informasi semester.
- Ijazah: berisi nama universitas, program studi, tahun kelulusan, dan predikat.
- Surat Keterangan: berisi jenis surat, NIM, dan tahun ajaran.
- Surat Resmi antar Fakultas: berisi informasi pengirim, penerima, dan perihal surat.

b. Apa tujuan hashing ini?

1. Menjaga Integritas Dokumen - Memastikan bahwa dokumen akademik yang diterbitkan tidak mengalami perubahan (baik disengaja maupun tidak). Setiap perubahan sekecil apapun pada isi dokumen akan menghasilkan nilai hash yang berbeda total, sehingga modifikasi dapat langsung terdeteksi.
2. Autentikasi Dokumen Resmi - Untuk dokumen yang dikirim antar pihak (misalnya surat dari Dekan ke Rektorat), hashing digunakan bersama kunci rahasia (HMAC) untuk membuktikan bahwa dokumen benar-benar berasal dari pengirim yang sah.
3. Bukti Keaslian (Digital Fingerprint) - Nilai hash berfungsi sebagai fingerprint digital dari dokumen. Pihak penerima dapat membandingkan hash yang dihitung ulang dengan hash yang dicatatkan sebelumnya.

c. Mengapa perlu Hashing?

1. Dokumen akademik bersifat krusial - Transkrip nilai dan ijazah menentukan karir seseorang. Pemalsuan dokumen (misalnya mengubah IPK) merupakan tindakan serius yang harus dapat dideteksi.
2. Fungsi satu arah (one-way function) - Dari nilai hash, tidak mungkin dikembalikan menjadi data aslinya. Ini menjamin bahwa penyerang tidak bisa merekonstruksi isi dokumen dari hash-nya.

3. Sensitivitas tinggi terhadap perubahan (avalanche effect) - Mengubah satu karakter saja (misalnya IPK dari "3.85" menjadi "3.95") akan menghasilkan hash yang sepenuhnya berbeda.
4. Output berukuran tetap - Berapapun panjang isi dokumen, hash SHA-256 selalu menghasilkan output 256-bit (64 karakter hex). Ini memudahkan penyimpanan dan perbandingan

Tugas 2

Metode: SHA-256 (dan HMAC-SHA256 untuk skenario autentikasi)

SHA-256 adalah varian dari keluarga SHA-2 (Secure Hash Algorithm 2) yang menghasilkan output hash sepanjang 256-bit (64 karakter heksadesimal)

Alasan pemilihan SHA-256:

Kriteria	Penjelasan
Keamanan	SHA-256 masih dianggap aman dan belum ditemukan serangan kolisi praktis, berbeda dengan SHA-1 yang sudah deprecated sejak 2017.
Standar Industri	Digunakan secara luas di TLS/SSL, Bitcoin, sertifikat digital X.509. NIST merekomendasikan SHA-256 sebagai standar hashing aman.
Performa	Keseimbangan baik antara keamanan dan kecepatan. Lebih efisien dibanding SHA-512 untuk data berukuran sedang.
Dukungan Library	Python menyediakan dukungan bawaan melalui modul hashlib yang sudah teruji dan optimal.
Panjang Output	Output 256-bit cukup aman untuk menghindari birthday attack dan cukup ringkas untuk disimpan sebagai fingerprint

Tugas 3

Jawaban: Ya, sebagian kasus membutuhkan penggunaan HMAC.

Skenario 1: Verifikasi Integritas Internal (Tanpa HMAC)

Untuk keperluan internal seperti mengecek apakah file transkrip telah dimodifikasi atau rusak, cukup menggunakan

SHA-256 biasa (tanpa kunci). Alasannya:

- Tujuannya hanya memastikan dokumen tidak berubah (integrity check).
- Tidak ada kebutuhan untuk membuktikan siapa yang membuat hash tersebut.
- Contoh: Membandingkan hash file setelah diunduh dengan hash asli di website universitas

Skenario 2: Autentikasi Dokumen Antar Pihak (Dengan HMAC)

Untuk dokumen yang dikirim antar pihak (misalnya surat rekomendasi dari Dekan ke Rektorat), diperlukan HMAC-SHA256. Alasannya:

- Perlu autentikasi pengirim: Penerima harus bisa membuktikan bahwa dokumen berasal dari pengirim yang sah.
- Perlindungan dari pemalsuan: Hash biasa bisa dihitung ulang oleh siapapun. Dengan HMAC, penyerang tidak bisa menghasilkan hash valid tanpa kunci rahasia.
- Kunci simetris dibagikan secara aman: Kedua pihak memiliki kunci rahasia yang sama

Tabel perbandingan Hash vs HMAC:

Aspek	Hash (SHA-256)	HMAC (HMAC-SHA256)
Input	Pesan saja	Pesan + kunci rahasia
Kegunaan	Cek integritas	Autentikasi + Integritas
Siapa bisa membuat?	Siapapun	Hanya pemilik kunci
Perlindungan dari	Perubahan tidak sengaja	Pemalsuan yang disengaja

Tugas 4

kode program

```
import hashlib
import hmac

# BAGIAN 1: Hashing dengan SHA-256 (Tanpa Kunci)
# Digunakan untuk: Verifikasi integritas dokumen

print("=" * 65)
print("  SISTEM VERIFIKASI INTEGRITAS DOKUMEN AKADEMIK")
print("  Menggunakan SHA-256 dan HMAC-SHA256")
print("=" * 65)

# Contoh Data Dokumen Akademik (Hardcoded)
dokumen_transkrip = "Nama: Budi Santoso | NIM: 13522001 | IPK: 3.85 | Semester: 8"
dokumen_ijazah = "Universitas XYZ | Program Studi: Informatika | Lulus: 2026 | Predikat: Cum Laude"
dokumen_surat = "Surat Keterangan Aktif | NIM: 13522001 | Tahun Ajaran: 2025/2026"
```

```

# Dokumen yang sudah dimodifikasi (untuk simulasi tampering)
dokumen_transkrip_palsu = "Nama: Budi Santoso | NIM: 13522001 | IPK:
3.95 | Semester: 8" # IPK diubah

print("\n" + "-" * 65)
print("BAGIAN 1: HASHING DENGAN SHA-256 (Verifikasi Integritas)")
print("-" * 65)

# Fungsi hashing SHA-256
def hash_sha256(data: str) -> str:
    """Menghitung hash SHA-256 dari string input."""
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

# Hitung hash untuk setiap dokumen
hash_transkrip = hash_sha256(dokumen_transkrip)
hash_ijazah = hash_sha256(dokumen_ijazah)
hash_surat = hash_sha256(dokumen_surat)
hash_transkrip_palsu = hash_sha256(dokumen_transkrip_palsu)

print("\n[1] Dokumen Transkrip")
print(f"    Data Asli   : {dokumen_transkrip}")
print(f"    SHA-256      : {hash_transkrip}")

print("\n[2] Dokumen Ijazah")
print(f"    Data Asli   : {dokumen_ijazah}")
print(f"    SHA-256      : {hash_ijazah}")

print("\n[3] Dokumen Surat Keterangan")
print(f"    Data Asli   : {dokumen_surat}")
print(f"    SHA-256      : {hash_surat}")

# Simulasi Verifikasi Integritas
print("\n" + "-" * 65)
print("SIMULASI VERIFIKASI INTEGRITAS DOKUMEN")
print("-" * 65)

print("\n[!] Dokumen Transkrip ASLI:")
print(f"    Data   : {dokumen_transkrip}")
print(f"    Hash   : {hash_transkrip}")

print("\n[!] Dokumen Transkrip PALSU (IPK diubah dari 3.85 ->
3.95):")

```

```

print(f"    Data    : {dokumen_transkrip_palsu}")
print(f"    Hash    : {hash_transkrip_palsu}")

if hash_transkrip == hash_transkrip_palsu:
    print("\n    >> HASIL: Hash SAMA - Dokumen TIDAK berubah
(identik)")
else:
    print("\n    >> HASIL: Hash BERBEDA - Dokumen TERDETEKSI telah
dimodifikasi!")
    print("    >> PERINGATAN: Integritas dokumen terganggu!")

# BAGIAN 2: HMAC-SHA256 (Dengan Kunci Rahasia)
# Digunakan untuk: Autentikasi dokumen antar pihak

print("\n\n" + "-" * 65)
print("BAGIAN 2: HMAC-SHA256 (Autentikasi dengan Kunci Rahasia)")
print("-" * 65)

# Kunci rahasia yang dibagikan antara Fakultas dan Rektorat
kunci_rahasia = b"KunciRahasiaFakultasRektorat2026!"

# Dokumen resmi yang dikirim
dokumen_resmi = "Surat Rekomendasi | Dari: Dekan Fakultas Informatika
| Untuk: Rektorat | Perihal: Beasiswa Mahasiswa Berprestasi"

def hitung_hmac_sha256(kunci: bytes, data: str) -> str:
    """Menghitung HMAC-SHA256 dari data dengan kunci rahasia."""
    return hmac.new(kunci, data.encode('utf-8'),
hashlib.sha256).hexdigest()

# Pengirim (Fakultas) menghitung HMAC
hmac_pengirim = hitung_hmac_sha256(kunci_rahasia, dokumen_resmi)

print("\n--- Sisi Pengirim (Fakultas) ---")
print(f"    Dokumen    : {dokumen_resmi}")
print(f"    Kunci       : {kunci_rahasia.decode()}")
print(f"    HMAC-SHA256: {hmac_pengirim}")

# Simulasi Penerima (Rektorat) memverifikasi
print("\n--- Sisi Penerima (Rektorat) ---")

# Dokumen diterima tanpa perubahan

```

```

hmac_penerima = hitung_hmac_sha256(kunci_rahasia, dokumen_resmi)
print("\n[Skenario 1] Dokumen diterima TANPA perubahan:")
print(f"    HMAC Pengirim : {hmac_pengirim}")
print(f"    HMAC Penerima : {hmac_penerima}")

if hmac.compare_digest(hmac_pengirim, hmac_penerima):
    print("    >> HASIL: HMAC COCOK - Dokumen AUTENTIK dan UTUH!")
else:
    print("    >> HASIL: HMAC TIDAK COCOK - Dokumen TIDAK autentik!")

# Dokumen dimodifikasi oleh pihak ketiga (man-in-the-middle)
dokumen_dimodifikasi = "Surat Rekomendasi | Dari: Dekan Fakultas
Informatika | Untuk: Rektorat | Perihal: Beasiswa Semua Mahasiswa"
hmac_dokumen_palsu = hitung_hmac_sha256(kunci_rahasia,
dokumen_dimodifikasi)

print("\n[Skenario 2] Dokumen DIMODIFIKASI oleh penyerang:")
print(f"    Dokumen Asli      : ...Perihal: Beasiswa Mahasiswa
Berprestasi")
print(f"    Dokumen Dimodifikasi: ...Perihal: Beasiswa Semua
Mahasiswa")
print(f"    HMAC Pengirim      : {hmac_pengirim}")
print(f"    HMAC Dok. Palsu    : {hmac_dokumen_palsu}")

if hmac.compare_digest(hmac_pengirim, hmac_dokumen_palsu):
    print("    >> HASIL: HMAC COCOK - Dokumen autentik")
else:
    print("    >> HASIL: HMAC TIDAK COCOK - Dokumen TERDETEKSI
dipalsukan!")

# Penyerang mencoba dengan kunci berbeda
kunci_palsu = b"KunciPalsuPenyerang12345678901!"
hmac_kunci_palsu = hitung_hmac_sha256(kunci_palsu, dokumen_resmi)

print("\n[Skenario 3] Penyerang menggunakan KUNCI BERBEDA:")
print(f"    Kunci Asli      : {kunci_rahasia.decode()}")
print(f"    Kunci Palsu     : {kunci_palsu.decode()}")
print(f"    HMAC Asli      : {hmac_pengirim}")
print(f"    HMAC Palsu     : {hmac_kunci_palsu}")

if hmac.compare_digest(hmac_pengirim, hmac_kunci_palsu):
    print("    >> HASIL: HMAC COCOK - Dokumen autentik")

```

```
else:
    print("    >> HASIL: HMAC TIDAK COCOK - Kunci tidak valid,
    autentikasi GAGAL!")

# RINGKASAN
print("\n\n" + "=" * 65)
print("  RINGKASAN")
print("=" * 65)
print("""
    1. SHA-256 (Hash biasa):
        - Digunakan untuk menjaga INTEGRITAS data
        - Perubahan sekecil apapun menghasilkan hash yang berbeda total
        - Tidak menggunakan kunci (siapapun bisa menghitung)

    2. HMAC-SHA256 (Hash + Kunci):
        - Digunakan untuk AUTENTIKASI + INTEGRITAS
        - Membutuhkan kunci rahasia yang sama di kedua pihak
        - Melindungi dari pemalsuan yang disengaja (man-in-the-middle)
    """)
print("=" * 65)
```

Output

```
=====
SISTEM VERIFIKASI INTEGRITAS DOKUMEN AKADEMIK
Menggunakan SHA-256 dan HMAC-SHA256
=====

-----
BAGIAN 1: HASHING DENGAN SHA-256 (Verifikasi Integritas)
-----

[1] Dokumen Transkrip
Data Asli : Nama: Budi Santoso | NIM: 13522001 | IPK: 3.85 | Semester: 8
SHA-256 : d02fd6da690104692820b60ec5a90bc25415d0cd68d2967d406101fdf26468ba

[2] Dokumen Ijazah
Data Asli : Universitas XYZ | Program Studi: Informatika | Lulus: 2026 | Predikat: Cum Laude
SHA-256 : ad8483b57b66ff14f26f885442fa55a4b4907dfea28427100861643e5eaf5f65

[3] Dokumen Surat Keterangan
Data Asli : Surat Keterangan Aktif | NIM: 13522001 | Tahun Ajaran: 2025/2026
SHA-256 : 1474bf461ba7ce89f9a3c4e8164d9d64e1dfa20beb23e55e3410134e09771717

-----
SIMULASI VERIFIKASI INTEGRITAS DOKUMEN
-----

[!] Dokumen Transkrip ASLI:
Data : Nama: Budi Santoso | NIM: 13522001 | IPK: 3.85 | Semester: 8
Hash : d02fd6da690104692820b60ec5a90bc25415d0cd68d2967d406101fdf26468ba

[!] Dokumen Transkrip PALSU (IPK diubah dari 3.85 -> 3.95):
Data : Nama: Budi Santoso | NIM: 13522001 | IPK: 3.95 | Semester: 8
Hash : 06e62853dad1e2e53efc21c661c67ad83643ab448112a8181203408f99338b1a

>> HASIL: Hash BERBEDA - Dokumen TERDETEKSI telah dimodifikasi!
>> PERINGATAN: Integritas dokumen terganggu!

-----
BAGIAN 2: HMAC-SHA256 (Autentikasi dengan Kunci Rahasia)
-----

--- Sisi Pengirim (Fakultas) ---
Dokumen : Surat Rekomendasi | Dari: Dekan Fakultas Informatika | Untuk: Rektorat | Perihal: Beasiswa Mahasiswa Berprestasi
Kunci : KunciRahasiaFakultasRektorat2026!
HMAC-SHA256: 890b5215716187e87c5686ee2d4b50f7f6979c89e16bc7dc4419fedd77eb332e

--- Sisi Penerima (Rektorat) ---

[Skenario 1] Dokumen diterima TANPA perubahan:
HMAC Pengirim : 890b5215716187e87c5686ee2d4b50f7f6979c89e16bc7dc4419fedd77eb332e
HMAC Penerima : 890b5215716187e87c5686ee2d4b50f7f6979c89e16bc7dc4419fedd77eb332e
>> HASIL: HMAC COCOK - Dokumen AUTENTIK dan UTUH!

[Skenario 2] Dokumen DIMODIFIKASI oleh penyerang:
Dokumen Asli : ...Perihal: Beasiswa Mahasiswa Berprestasi
Dokumen Dimodifikasi: ...Perihal: Beasiswa Semua Mahasiswa
HMAC Pengirim : 890b5215716187e87c5686ee2d4b50f7f6979c89e16bc7dc4419fedd77eb332e
HMAC Dok. Palsu : aac5606b4144cfccf4ceb5f69acd57d31b7b4318f307ac791789489ce2cc065
>> HASIL: HMAC TIDAK COCOK - Dokumen TERDETEKSI dipalsukan!

[Skenario 3] Penyerang menggunakan KUNCI BERBEDA:
Kunci Asli : KunciRahasiaFakultasRektorat2026!
Kunci Palsu : KunciPalsuPenyerang12345678901!
HMAC Asli : 890b5215716187e87c5686ee2d4b50f7f6979c89e16bc7dc4419fedd77eb332e
HMAC Palsu : 553a60461e5eb0ce71c316d747dc06c862a994657c4f279a2df5705dff6c6db1
>> HASIL: HMAC TIDAK COCOK - Kunci tidak valid, autentikasi GAGAL!
```



```
=====
RINGKASAN
=====
```

- ```
1. SHA-256 (Hash biasa):
- Digunakan untuk menjaga INTEGRITAS data
- Perubahan sekecil apapun menghasilkan hash yang berbeda total
- Tidak menggunakan kunci (siapa pun bisa menghitung)

2. HMAC-SHA256 (Hash + Kunci):
- Digunakan untuk AUTENTIKASI + INTEGRITAS
- Membutuhkan kunci rahasia yang sama di kedua pihak
- Melindungi dari pemalsuan yang disengaja (man-in-the-middle)
=====
```

## Tugas 5

### Alur program secara keseluruhan

Program terdiri dari dua bagian utama yang mensimulasikan dua skenario penggunaan hashing pada sistem verifikasi dokumen akademik.

#### Bagian 1: Hashing dengan SHA-256 (Tanpa Kunci)

Alur: Data Dokumen (String) -> Encode ke UTF-8 (Bytes) -> Fungsi SHA-256 -> Hash Value (Hex String 64 karakter)

1. Definisi Data - Tiga dokumen akademik (transkrip, ijazah, surat keterangan) didefinisikan sebagai string hardcoded. Satu dokumen palsu juga dibuat (transkrip dengan IPK diubah dari 3.85 menjadi 3.95).
2. Proses Hashing - Fungsi `hash_sha256()` menerima string input, mengubahnya menjadi bytes menggunakan encoding UTF-8, kemudian menghitung hash SHA-256 menggunakan `hashlib.sha256()`. Hasilnya dikembalikan dalam format heksadesimal menggunakan `.hexdigest()`.
3. Verifikasi Integritas - Program membandingkan hash dari dokumen asli dengan hash dari dokumen yang telah dimodifikasi. Karena SHA-256 memiliki sifat avalanche effect, perubahan IPK menghasilkan hash yang sepenuhnya berbeda, sehingga pemalsuan terdeteksi

#### Bagian 2: HMAC-SHA256 (Dengan Kunci Rahasia)

Alur: Kunci Rahasia (Bytes) + Data (String) -> Encode UTF-8 -> Fungsi HMAC-SHA256 -> HMAC Value (Hex 64 karakter)

1. Definisi Kunci dan Dokumen - Sebuah kunci rahasia didefinisikan sebagai bytes, dan dokumen resmi didefinisikan sebagai string.
2. Penghitungan HMAC oleh Pengirim - Fungsi `hitung_hmac_sha256()` menggunakan modul `hmac` Python. Fungsi `hmac.new()` menerima tiga parameter: kunci rahasia (bytes), data (bytes), dan algoritma hash (`hashlib.sha256`).
3. Verifikasi oleh Penerima - Tiga skenario diuji: (a) Dokumen tidak berubah -> HMAC cocok -> Dokumen autentik. (b) Dokumen dimodifikasi -> HMAC tidak cocok -> Pemalsuan terdeteksi. (c) Kunci berbeda -> HMAC tidak cocok -> Autentikasi gagal.

4. Perbandingan Aman - Digunakan `hmac.compare_digest()` yang tahan terhadap serangan timing attack